

Falta de constancia en hábitos de actividad física

CC3091 – Ingeniería de Software 2

Camila Sandoval **#24358**

Emily Góngora **#24308**

Alejandra Sierra **#24405**

Martin Villatoro **#24033**

Alejandro Pérez **#24136**

Esteban de la Peña **#24171**

Link a repositorio backend: <https://github.com/mariale-sierra/backend.git>

Link a repositorio frontend: <https://github.com/mariale-sierra/frontend.git>

Link a repositorio de documentación proyecto:
https://github.com/mariale-sierra/Proyecto_IngSoftware.git

Introducción

El presente documento resume el trabajo realizado durante el Sprint 8 del proyecto Havit, desarrollado del 17 al 29 de septiembre. Durante este periodo, el equipo se enfocó en completar funcionalidades administrativas, implementar mecanismos de moderación, mejorar la experiencia de los usuarios nuevos, corregir problemas de concurrencia y optimizar el rendimiento del backend mediante Redis. Además, se ejecutaron pruebas unitarias, de seguridad, volumen y usabilidad para validar el funcionamiento de la aplicación, identificar oportunidades de mejora y definir las tareas prioritarias para el próximo sprint.

Objetivos del Sprint

- Completar las funcionalidades administrativas pendientes, implementando las vistas, permisos y herramientas correspondientes para el administrador y el dueño de cada challenge.
- Fortalecer la seguridad de la comunidad mediante la moderación de fotografías y textos, el reporte de contenido inapropiado y la aplicación de penalizaciones.
- Mejorar la experiencia y confiabilidad de la aplicación mediante un onboarding para usuarios nuevos y la corrección del problema de progreso duplicado causado por solicitudes simultáneas.
- Optimizar el rendimiento y la escalabilidad del backend utilizando mejoras en PostgreSQL y Redis, además de preparar los protocolos y ejecutar las pruebas de Neurolab.

Product Backlog

Hasta el Sprint 7, el Product Backlog de Havit se ha enfocado principalmente en desarrollar y consolidar el flujo central de la aplicación. Se han trabajado funcionalidades relacionadas con autenticación y usuarios, creación y exploración de challenges, configuración de ciclos y días de descanso, rutinas y ejercicios con métricas dinámicas, ejecución y registro de workouts, seguimiento del progreso mediante fotografías, publicaciones asociadas a los challenges y gestión del perfil. Además, se incorporaron funcionalidades como invitaciones a challenges, privacidad de publicaciones, moderación de contenido, seguidores y visualización de publicaciones de otros usuarios.

Paralelamente, el backlog ha incluido actividades orientadas a mejorar la calidad y mantenibilidad del producto, como limpieza y refactorización del código, pruebas unitarias, documentación y mejoras de interfaz y experiencia de usuario. Con estos avances, Havit cuenta con un flujo funcional que conecta

Challenge → Rutina → Workout → Progreso → Publicación → Perfil → Interacción social

Dejando como trabajo posterior funcionalidades más avanzadas como administración, reportes y penalizaciones, optimización y escalabilidad, onboarding, notificaciones, chats y Spaces.

Sprint Backlog

Para el Sprint 8 se organizó el trabajo mediante bloques funcionales. Cada bloque concentra el desarrollo de un módulo específico y asigna un responsable principal para reducir dependencias entre integrantes.

Bloque 1 (17 septiembre - 20 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción
B1: Vista de administrador y dueño de challenge	Ale Sierra	10 h	8	Completar la deuda técnica relacionada con las vistas de administrador y dueño de challenge. Incluye visualizar información administrativa, gestionar participantes, acceder a opciones exclusivas según el rol, validar permisos y comprobar ambos flujos desde el frontend.

Bloque 2 (17 septiembre - 21 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción
B2: Moderación manual de fotografías, reportes y penalizaciones	Emily	10 h	8	Implementar el flujo completo de moderación manual de fotografías. Incluye reportar contenido, revisar reportes, aprobar o rechazar fotografías, aplicar penalizaciones, actualizar el estado del contenido y mostrar al usuario el resultado de la moderación.

Bloque 3 (19 septiembre - 22 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción
B3: Moderación automática de texto	Martín	6 h	5	Identificar los campos de texto moderables, crear un servicio reutilizable e integrarlo en la creación y edición de contenido. También incluye estandarizar los errores, mostrar retroalimentación al usuario, probar contenido aceptado y rechazado y documentar en qué módulos se aplica la moderación.

Bloque 4 (17 septiembre - 23 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción
B4: Flujo de onboarding	Camila	10 h	8	Definir el recorrido del usuario nuevo, diseñar e implementar las pantallas de onboarding y agregar navegación para avanzar, regresar u omitir. Incluye persistir que el proceso fue completado, integrarlo con registro e inicio de sesión, diferenciar usuarios nuevos de existentes y validar el flujo completo.

Bloque 5 (25 - 26 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción
B5: Concurrencia e integridad de Workout Progress	Esteban	10 h	8	Reproducir el problema de solicitudes simultáneas en /workout-logs/progress, definir la regla de unicidad del progreso diario e incorporar protección a nivel de base de datos. Incluye ajustar el endpoint, manejar conflictos o transacciones, crear pruebas concurrentes y validar escenarios de doble toque y reintentos desde el frontend.

Bloque 6 (27 - 28 septiembre)

Tarea	Encargado	Duración	Story Points	Descripción

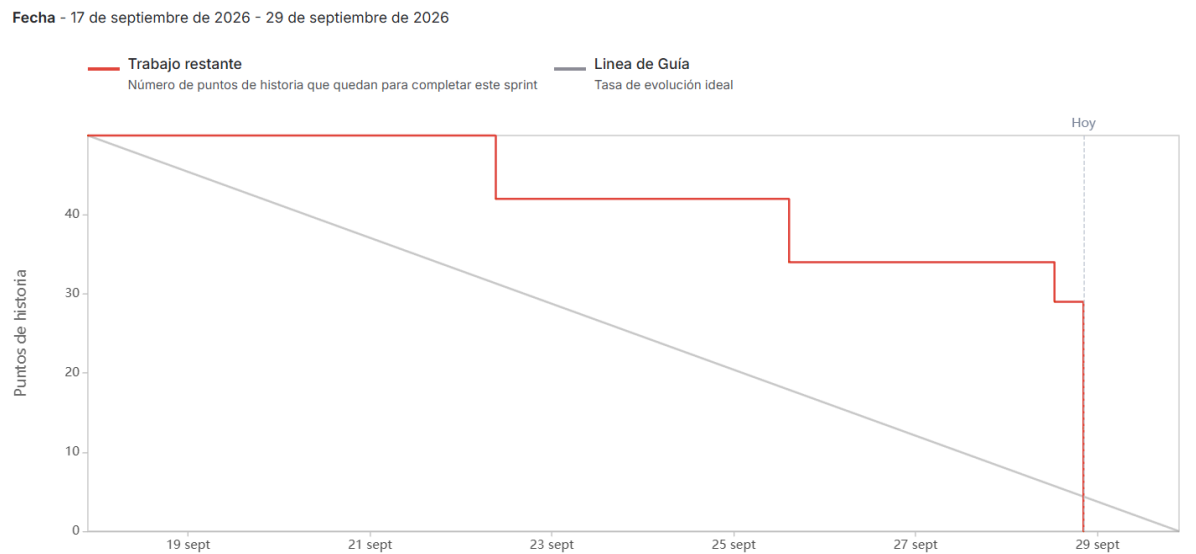
B6: Rendimien to, Redis y escalabilid ad	Ale Pérez	16 h	13	Establecer un baseline de rendimiento, definir endpoints y niveles de concurrencia y medir tiempos de respuesta, errores y throughput. Incluye identificar consultas problemáticas, revisar índices de PostgreSQL, optimizar consultas críticas, configurar Redis, implementar caché con TTL e invalidación, evaluar rate limiting y comparar los resultados antes y después de las mejoras.
--	-----------	------	----	--

Resultados del Sprint

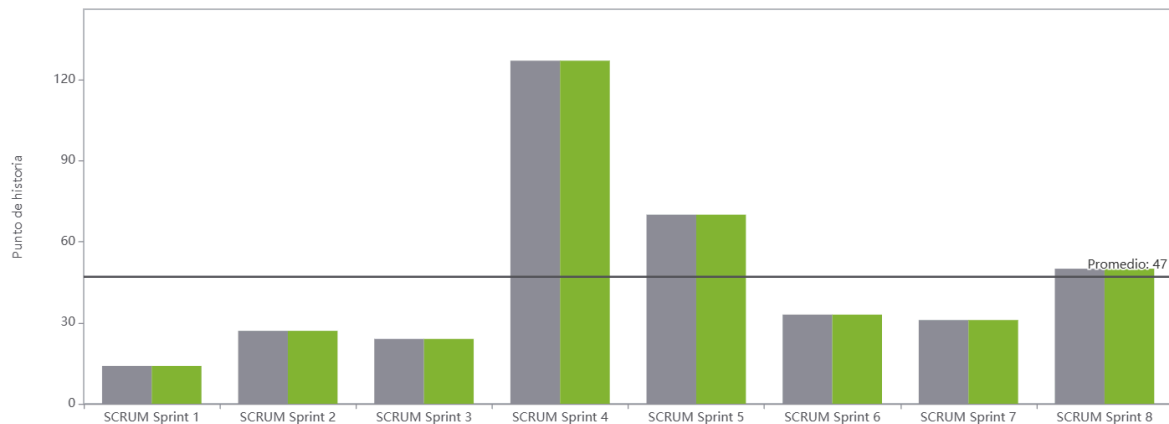
Uso de Jira

SCRUM Sprint 8 17 sep – 29 sep (6 actividades)				42	8	0	Completar sprint	...
☒	SCRUM-141	B1 Ale S: Admin view y roles	En curso	22 sept	8	=	MC	
☒	SCRUM-142	B2 Emily: Moderacion manual, reportes y penalizaciones	Tareas por hacer	23 sept	8	=	EG	
☒	SCRUM-143	B3 Martin: Moderacion de texto	Tareas por hacer	24 sept	5	=	MG	
☒	SCRUM-144	B4 Cami: Onboarding de nuevos usuarios y guias	Tareas por hacer	25 sept	8	=	CC	
☒	SCRUM-145	B5 Esteban: Corregir endpoint workout progress	Tareas por hacer	26 sept	8	=	EG	
☒	SCRUM-146	B6 Ale P: Mejorar rendimiento backend (redis)	Tareas por hacer	28 sept	13	>	AM	
+ Crear								

Gráfica Burndown



Gráfica de velocidad



La gráfica burndown muestra que el avance del sprint no fue constante, ya que durante los primeros días el trabajo pendiente se mantuvo sin cambios y la mayor reducción de puntos ocurrió por bloques, especialmente cerca del cierre. Aunque el equipo se mantuvo por encima de la línea ideal durante casi todo el sprint, finalmente logró completar todos los puntos planificados dentro del plazo. Por su parte, la gráfica de velocidad muestra que el Sprint 8 alcanzó aproximadamente 50 puntos, superando el promedio histórico de 47 y mejorando frente a los Sprints 6 y 7. Esto evidencia una recuperación en la capacidad de entrega del equipo, aunque también señala la necesidad de distribuir y cerrar las tareas de forma más uniforme durante el sprint.

Pruebas UX

Para realizar las pruebas de experiencia de usuario se elaboraron dos formularios complementarios: uno dirigido al participante y otro al moderador. El formulario del participante permitió conocer si logró completar cada actividad, qué comprendió del flujo, qué dificultades experimentó y qué tan fácil le resultó mediante una escala de facilidad del 1 al 7. Por su parte, el formulario del moderador se utilizó para registrar datos observables, como el éxito sin ayuda, el tiempo empleado, los errores, retrocesos, solicitudes de ayuda y comentarios realizados durante la prueba. Esta metodología permitió comparar lo que el usuario percibió con su comportamiento real al utilizar Havit e identificar los flujos que requieren mejoras. A continuación, se incluyen los enlaces a ambos formularios para consultar las tareas y preguntas utilizadas.

Forms:

FORMULARIO DEL PARTICIPANTE (para responder):

https://docs.google.com/forms/d/e/1FAIpQLSeVk4bXeBAn_J4Z1-e_EPQOYS1okivrlNBHsLUWiw5_8lclZQ/viewform

FORMULARIO DEL MODERADOR (para responder):

<https://docs.google.com/forms/d/e/1FAIpQLSdEy0hQRB4nCuvWsq1-r0rqHXh9SgmVT3v5lCl1ndWqPKgg/viewform>

RESPUESTAS DE PARTICIPANTES:

<https://docs.google.com/spreadsheets/d/1jf5Xqd0V3AG0cLRVNImQdP49rBXcz95WJWi4YJGMzFc/edit>

RESPUESTAS DE MODERADORES:

https://docs.google.com/spreadsheets/d/1z8XRj4br8AWVIIUHa_B3aVHMQawOxEgKeQzsOndq_So/edit

Las actividades 2 (Challenge) y 4 (Spaces) son las más críticas: solo 2 de 12 las completaron sin ayuda y concentran todos los casos parciales o no completados.

Actividad	Éxito sin ayuda	Parcial / No completada	Tiempo promedio (mm:ss)	Solicitudes de ayuda	Facilidad (1-7)
1. Onboarding y registro	7/12	0	2:59	2	5.8
2. Crear o unirse a un Challenge	2/12	3	3:36	6	4.3
3. Registro diario de progreso	4/12	0	2:11	4	4.8
4. Descubrimiento y unión a Spaces	2/12	3	3:15	8	4.0
5. Feed social y perfil	7/12	0	1:36	2	5.8
6. Mensajería y navegación al perfil	5/12	0	1:33	6	5.2

Implementación en el próximo sprint

El sprint prioriza los tres cambios que bloquean los flujos de Challenges y Spaces, y valida su impacto con una nueva ronda de pruebas.

Prioridad alta (bloquean flujos principales)

- Explicar Space vs. Challenge: tooltip o tarjeta explicativa la primera vez que se entra a cada sección; diferenciarlos visualmente con íconos, colores y copy propios.
- Rediseñar el estado "Pendiente": copy explícito ("Solicitud enviada, el admin del Space debe aprobarla"), botón deshabilitado mientras está pendiente y notificación al aprobarse.
- Confirmación al unirse a un Challenge: toast o modal de éxito y redirección a la rutina del día; subir el botón "Unirme" para que se vea sin hacer scroll.

Prioridad media

- Registro diario: cambiar el botón a "Registrado hoy ✓" cuando el día ya está registrado; aclarar si la foto es opcional u obligatoria.
- Navegación chat ↔ perfil: que "atrás" regrese siempre a la conversación y conserve su estado.

Prioridad baja

- Copy en el onboarding que explique para qué se usan las actividades elegidas.
- Poder tocar un badge para ver cómo se obtuvo.
- Revisar qué términos en inglés conviene traducir o explicar.

Cómo lo medimos: repetir las actividades 2 y 4 con 5 a 6 usuarios nuevos después de los cambios. La meta es que al menos 8 de 12 las completen sin ayuda, con una facilidad de 5.5 / 7 o más.

Pruebas de Seguridad y Volumen con Requisitos No funcionales

Herramientas seleccionadas

Para volumen/inundación se usó HammerDB porque de las cuatro sugeridas es la única diseñada para generar carga y concurrencia real sobre bases de datos relacionales, DbFit y jdbcslim son frameworks de pruebas funcionales (no de carga), y NoSQLMap apunta a bases NoSQL, que Havit no usa (el backend usa PostgreSQL vía TypeORM).

Para seguridad se usó OWASP ZAP porque de las herramientas es la única orientada a auditar aplicaciones web/APIs REST siguiendo el OWASP Top 10, con soporte para importar el spec OpenAPI de Havit y probar activamente cada endpoint real.

Resultados de Inundación

VU (concurrencia)	NOPM	TPM	Resultado
10	25,929	59,861	Estable

50	52,958,1E7B34	121,326,1E7B34	Pico de throughput, 1E7B34
100	20,587,B3261E	45,685,B3261E	Colapso (-60%),B3261E
150	20,585,B3261E	47,516,B3261E	Se mantiene colapsado,B3261E

El sistema colapsa su rendimiento entre 50 y 100 conexiones concurrentes (PostgreSQL con “max_connections=100” por defecto) muy lejos de la meta que establecimos en los requisitos funcionales (“al menos 5000”). No es una falla de la aplicación (no hay caídas ni corrupción de datos), sino de configuración de infraestructura.

Resultados de Seguridad

- Escaneo pasivo (baseline): 63 pruebas superadas, 4 advertencias menores, 0 fallas.
- Escaneo activo (SQLi, XSS, RCE, XXE, SSRF, etc.): 118 pruebas superadas, 1 advertencia menor, 0 fallas.
- Ningún hallazgo de severidad alta o crítica. Los 5 hallazgos menores son de cabeceras HTTP (CSP con “unsafe-inline”, falta “Permissions-Policy”, falta “Cross-Origin-Embedder-Policy”, “Content-Type” inconsistente en la raíz, contenido cacheable en rutas públicas).

Cómo se van a solucionar los problemas encontrados

- Concurrencia: introducir un pool de conexiones (PgBouncer) entre backend y PostgreSQL, ajustar “max_connections” en Azure, y mover el rate-limiter de memoria a Redis compartido antes de escalar a más de una instancia.
- Security headers: ajustar la configuración de “helmet()” en “src/main.ts” para endurecer CSP y agregar “Permissions-Policy” y “Cross-Origin-Embedder-Policy”.
- Vacíos legales: implementar pantalla de aceptación de términos y condiciones en el registro, y un endpoint de eliminación de cuenta/datos con auditoría.

Comparación con los requisitos no funcionales del diseño

Categoría NFR	Estado	Resumen
Rendimiento	Parcial	Tiempos de respuesta (login/foto/pantalla) cumplen concurrencia ≥ 5000 NO cumple (colapsa en ~100).
Seguridad	Cumple	0 vulnerabilidades altas/críticas, autenticación y autorización por defecto verificadas activamente.
Portabilidad	Cumple (servidor)	Backend corre 100% en Docker/Linux, confirmado por el propio entorno de prueba

Políticos / Culturales	Cumple	Moderación automática cada 10 min (límite: 24h), unidades e idioma configurables en código.
Legal	No cumple	No existe flujo de términos y condiciones ni de eliminación de cuenta, ni en backend ni en frontend.
Interfaz interna	Cumple	Sin errores de integridad en las 129 rutas activas probadas por ZAP.
Software	Parcial	PostgreSQL 16 confirmado. Compatibilidad Android/iOS no evaluada en este bloque.

¿Qué pruebas no funcionales faltaría hacer?

- Usabilidad con usuarios reales: ese requisito se evaluará con las Pruebas UX y Neurolab para poder validar el flujo del sistema y la interfaz.
- Rendimiento contra Azure real (no solo local), y disponibilidad medida con monitoreo continuo durante al menos un mes.
- Prueba real de backup/restore (RPO/RTO)
- Volumen a mayor escala tras ajustar la infraestructura (PgBouncer), más una prueba de carga a nivel de API completa no solo de base de datos.
- Compatibilidad real en dispositivos Android 10 / iOS 15, y medición de RAM/almacenamiento.
- Revisión legal formal de cumplimiento normativo de privacidad.

¿Qué tanto se han cumplido los requisitos no funcionales?

De los 34 requisitos no funcionales planteados en el diseño, este bloque evaluó directamente 12 y 6 que no están relacionados con volumen y seguridad pero que de igual manera se trabajaron en este sprint, dando un total de 18 pruebas funcionales trabajadas:

- Cumplen (6): autenticación obligatoria (correo + contraseña cifrada), backend desplegable en Linux/Docker/nube pública, PostgreSQL 14+ (confirmado PostgreSQL 16), moderación de contenido en $\leq 24h$ (corre cada 10 min), visibilidad de publicaciones (control dedicado en código) y unidades de medida configurables.
- No cumplen (3): capacidad y concurrencia ≥ 5000 , aceptación de términos y condiciones (no existe ningún flujo, ni en backend ni en frontend) y eliminación de cuenta/datos en ≤ 7 días (no existe el endpoint).
- Cumplimiento parcial (3): tiempos de respuesta cumplen en el entorno local, pero no confirmados contra la latencia real de Azure, cifrado de información personal (las contraseñas sí usan bcrypt y Azure cifra en reposo, pero no hay cifrado explícito a nivel de columna para correo/fotos/métricas), e integración con API externa (OpenAI y Cloudflare R2 están integradas, pero no se midió el 95% de éxito)
- Ayudas y documentación (3): sección de ayuda, tutorial inicial, FAQ, esta área queda parcialmente completa con la implementación del módulo de onboarding y con las

pruebas del Neurolab podemos complementar qué tanto necesitamos secciones de ayuda y FAQ

- Soporte (3): cobertura de pruebas unitarias $\geq 80\%$, adaptabilidad de dificultad/pantalla, configuración de notificaciones y panel admin - este requisito esta parcialmente cubierto con la implementación del plan de pruebas desde sprint pasados y el panel de admin implementado en este sprint, quedan pendiente las notificaciones.

Los otros 16 requisitos no tienen todavía ninguna prueba no funcional dedicada porque abarcan áreas de diseño, interfaz, usabilidad, y otras falta implementar los módulos correspondientes como el área legal y portabilidad

- Apariencia (3): paleta neutra/lenguaje inclusivo, tiempo de renderizado $< 4s$, consistencia visual.
- Usabilidad (3): navegación sin ayuda, satisfacción y primer reto sin asistencia - queda parcialmente completa con las pruebas de UX que demuestran datos de satisfacción y navegación sin ayuda para mejorar en el siguiente sprint
- Rendimiento (1): disponibilidad $\geq 99\%$ mensual (requiere monitoreo continuo, no una prueba puntual).
- Portabilidad (1): funcionamiento en Android/iOS.
- Legal (1): cumplimiento de normativa local de privacidad (requiere revisión legal, no técnica).
- Confiabilidad (1): backups automáticos cada 24h.
- Interfaz interna (1): sincronización de datos entre módulos $\leq 5s$.
- Software/Hardware/Restricciones de diseño (5): compatibilidad de dispositivos, RAM/almacenamiento, conexión estable, exclusividad móvil/orientación vertical/sensores.

¿Fueron bien diseñados los requisitos no funcionales?

La categorización general es adecuada, pero varios requisitos individuales tienen problemas de medibilidad que esta prueba evidenció en la práctica:

- “Al menos 5000” no especifica la unidad (¿usuarios?, ¿requests/segundo?, ¿conexiones a BD?), generó ambigüedad real al diseñar la prueba de HammerDB.
- “Disponibilidad 99% mensual” no define cómo ni con qué herramienta se mide.
- “Correo y un código o contraseña encriptada” es ambiguo sobre si son alternativas o parte del mismo flujo.
- “Backups cada 24h” no define RPO/RTO ni cómo se comprobaría un restore exitoso.

ZAP Report:



Site: <http://backend:3000>
Generated on Tue, 29 Sept 2026 14:22:25
ZAP Version: 2.17.0
ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	2
Informational	1
False Positives	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://backend:3000	Percentage of responses with status code 2xx	50 %
Info	Informational	http://backend:3000	Percentage of responses with status code 4xx	50 %
Info	Informational	http://backend:3000	Percentage of endpoints with content type application/json	100 %
Info	Informational	http://backend:3000	Percentage of endpoints with method GET	100 %
Info	Informational	http://backend:3000	Count of total endpoints	2
Info	Informational	http://backend:3000	Percentage of slow responses	75 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
CSP: Wildcard Directive	Medium	1
CSP: style-src unsafe-inline	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Permissions Policy Header Not Set	Low	1
Storable and Cacheable Content	Informational	3

Alert Detail

Refactorización

En el Sprint pasado establecimos la deuda técnica del proyecto, y nos planteamos cubrirla durante el Sprint 7 y este, logramos mejorar las áreas de:

- Frontend y diseño
- Onboarding y selección de ejercicios
- Moderación de contenido
- Privacidad y fotografías
- Estructura y mantenimiento de código

De estas sólo quedó pendiente las pruebas de Neurolab para poder terminar todas las mejoras al frontend y diseño, realizamos los protocolos y preparamos el material el cual ya se encuentra aprobado para la sesión programada para el 8 de octubre. Además quedó

pendiente establecer los términos y condiciones de la aplicación para que el usuario las acepte antes de subir sus imágenes.

Para este Sprint identificamos estas áreas de mejora para trabajar estas semanas y en el siguiente:

Área de deuda técnica	Mejora o técnica de refactorización	Momento de realización
Escalabilidad y conexiones de base de datos	Introducir un pool de conexiones (PgBouncer) entre el backend y PostgreSQL, revisar max_connections en la instancia de Azure, y migrar el rate-limiter de @nestjs/throttler (actualmente en memoria, por proceso) a almacenamiento compartido en Redis para que el límite sea real al escalar a más de una instancia.	Sprint actual: ajuste de max_connections y pool de conexiones (hallazgo crítico de la prueba de volumen con HammerDB, el sistema colapsa en aprox 100 conexiones concurrentes, muy por debajo de la meta de 5000). Siguiente sprint: throttler compartido en Redis, antes de cualquier despliegue con más de una instancia del backend.
Security Headers	Ajustar la configuración de helmet() en main.ts: eliminar unsafe-inline de la directiva style-src, agregar Permissions-Policy y Cross-Origin-Embedder-Policy, y estandarizar el Content-Type de todas las respuestas (incluida la raíz /).	Sprint actual, junto con otras correcciones menores, bajo esfuerzo, identificado por el escaneo activo de OWASP ZAP.
Cumplimiento legal y gestión de datos personales (backend)	Implementar endpoint de aceptación de términos y condiciones con registro de auditoría (terms_accepted_at), y endpoint de eliminación/anonimización de cuenta (DELETE /users/me) que cumpla el plazo de 7 días exigido por la NFR legal.	Siguiente sprint, en coordinación con la iniciativa de "Privacidad y fotografías" ya planeada en frontend, hoy no existe ningún flujo de términos ni de eliminación de cuenta en ninguno de los dos lados.
Paginación de endpoints sin límite	Paginar GET /challenges y GET /workout-logs,	Siguiente sprint, una vez estabilizada la rama que

	identificados en el propio equipo (Sprint B6) como las consultas más lentas y sin límite de resultados — requiere coordinar el cambio de contrato con frontend.	modifica roles de challenges (dependencia ya documentada por el equipo).
Cobertura de pruebas no funcionales	Incorporar pruebas de carga a nivel de API (k6/Artillery) al flujo de trabajo, monitoreo continuo de disponibilidad, prueba real de backup/restore, y automatizar un escaneo de ZAP recurrente (por ejemplo en CI) en lugar de una prueba puntual manual.	Iterativo, a partir del sprint siguiente, una vez resueltos los hallazgos críticos de escalabilidad y legales de esta prueba.

Presupuesto

Presupuesto consumido:

Hasta la finalización del Sprint 8, se estima que se ha utilizado aproximadamente el 81.5% del costo total planificado del proyecto. Considerando un costo base estimado de Q331,305.57, el gasto acumulado asciende aproximadamente a Q270,015.04. Esto significa que quedan alrededor de Q61,290.53, equivalentes al 18.5% del presupuesto restante, para cubrir las actividades correspondientes a los últimos dos sprints. Este porcentaje refleja que el proyecto se encuentra en una etapa avanzada de ejecución, donde la mayor parte de los costos asociados al desarrollo e implementación ya han sido consumidos.

Tiempo consumido:

El proyecto contempla una estimación total de aproximadamente 1,100 horas de trabajo. Al cierre del Sprint 8, se calcula que se han utilizado alrededor de 896.5 horas, lo que representa aproximadamente el 81.5% del tiempo total estimado. Por lo tanto, quedan cerca de 203.5 horas, correspondientes al 18.5% restante, para completar las tareas pendientes en los Sprint 9 y 10. Esta distribución permite mantener un margen de tiempo para actividades finales como ajustes, pruebas, corrección de errores, validaciones y cierre del proyecto.

Retrospectiva del Sprint

¿Qué hicimos bien?

- Logramos realizar varias pruebas que nos dieron insights importantes sobre nuestro proyecto y de aspectos que podemos mejorar.

- Los módulos desarrollados terminan la implementación de los requisitos de la aplicación, lo que nos queda es reducir la deuda técnica y seguir realizando pruebas entre usuarios.
- Planificamos las tareas con tiempo para poder aprovechar las semanas que teníamos

¿Qué hicimos mal?

- Nos costó retomar la proactividad de la semana de descanso y eso hizo que las tareas principales se atrasaran y afectó el rendimiento del resto

¿Cómo vamos a mejorar?

- Creemos que después de la semana de descanso vamos a retomar y mantener mejor el ritmo de trabajo que hemos llevado

Conclusiones

- Avanzamos en la administración, moderación y onboarding de Havit, reduciendo parte de la deuda técnica.
- Las pruebas de usabilidad evidenciaron dificultades en Challenges y Spaces, por lo que se debe mejorar la claridad y navegación de estos flujos.
- Aunque las pruebas de seguridad no detectaron vulnerabilidades graves, aún es necesario mejorar la escalabilidad e implementar los procesos legales pendientes.

Formularios LOGT

Nombre: Alejandra Sierra

Carné: 24405

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Fase	Comentarios
20/09	17:00	18:00	10min	50min	Sprint 8	Correcciones de moderación y seleccionar herramienta para pruebas seguridad y volumen

21/09	21:00	23:00	1 hr	1h	Sprint 8	Vista de admin y challenge owner
29/09	08:00a m	10:00a m	30min	1h 30min	Sprint 8	Pruebas de seguridad y volumen, ver resultados de pruebas UX
29/09	10:00a m	12:00p m	-	2h	Sprint 8	Pruebas UX

Nombre: Martin Villatoro

Carne: 24033

Fecha	Inicio	Fin	Tiempo de interrupción	Delta de tiempo	Fase	Comentarios
26/9	21:30	23:00	0	1.5 h	Sprint 8	Análisis del flujo actual de moderación y definición de los campos de texto a validar.
26/9	21:00	23:30	0.5 h	2 h	Sprint 8	Implementación de la moderación automática en challenges,

						comentarios y biografía del usuario.
27/9	19:30	22:00	0.5 h	2 h	Sprint 8	Pruebas de contenido aceptado y rechazado, ajustes de feedback en frontend y documentación de la moderación.

Nombre: Emily Gongora

Carné: 24308

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Fase	Comentarios
27/09	18:00	21:20	20 min	3 horas	Sprint 8	Reportes de posts y comentarios, revisión de admin y strikes al autor
27/09	20:00	21:20		1 hr 20 min	Sprint 8	Botón de reportar en posts y comentarios y pantalla de moderación para admins

27/09	19:30	21:30	30 min	1 hr y 30 min	Sprint 8	Pruebas del flujo de reportes y documenta la moderación manual
-------	-------	-------	--------	---------------	----------	--

Nombre: Esteban de la Peña

Carné: 24171

Fecha	Inicio	Fin	Tiempo de interrupción	Delta de tiempo	Fase	Comentarios
26/09	14:00	18:00	30 min	3h 30 min	Sprint 8	Reproducción y análisis del problema de solicitudes simultáneas en /workout-logs/progress; definición de la regla de unicidad del progreso diario.
27/09	09:00	13:00	30 min	3h 30 min	Sprint 8	Implementación de la migración 2026-09-28-01-workout-logs-progress-uniqueness.sql con índice único parcial y manejo transaccional en workout-log.service.ts para impedir progreso duplicado y manejar conflictos de

						unicidad.
27/09	15:00	18:30	30 min	3h	Sprint 8	Creación de workout-log.service.concurrency.spec.ts y validación del endpoint bajo solicitudes concurrentes para comprobar que solo quede un registro válido de progreso.

Nombre: Camila Sandoval

Carne: 24358

Fecha	Inicio	Fin	Tiempo Interrupción	Delta Tiempo	Fase	Comentarios
24/09	19:00	21:00	—	2h	Sprint 7	Implementación de pantallas de onboarding en frontend (app/onboarding/practices.tsx).
25/09	18:00	20:30	30 min	2h	Sprint 7	Desarrollo de componentes de guía CoachMark.tsx y WelcomeGlowBackground.tsx.
27/09	20:00	22:00	—	2h	Sprint 7	Integración del flujo de onboarding con registro e inicio de sesión.

Nombre: Alejandro Pérez

Carné: 24136

Fecha	Inicio	Fin	Tiempo de interrupción	Delta de tiempo	Fase	Comentarios
25/09	18:30	23:00	30 min	4 h	Sprint 8	Implementación y configuración de Redis para mejorar el rendimiento del backend.
28/09	20:00	22:00	0	2 h	Sprint 8	Ejecución y validación de las pruebas unitarias desarrolladas durante el sprint.
29/09	09:00	11:00	0	2 h	Sprint 8	Apoyo en la ejecución de las pruebas de seguridad con OWASP ZAP